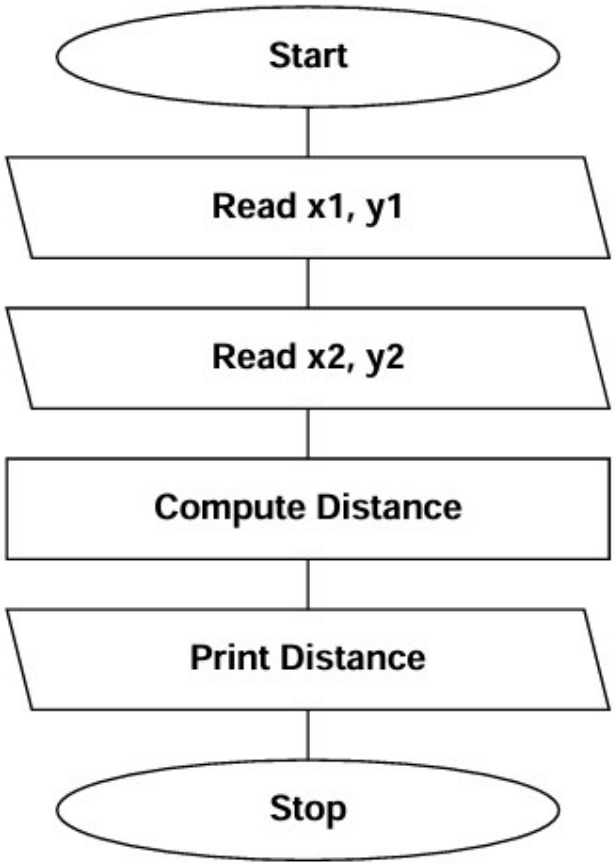
1. **A robot needs to find how far it must travel between two points on a 2D plane. Develop a C Program: to calculate the straight-line distance between the given coordinates.**

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Algorithm:

1. Start
2. Read x1, y1
3. Read x2, y2
4. Calculate distance
5. Display distance
6. Stop

Flowchart:



Program:

```
#include <stdio.h>
#include <math.h>

int main() {
   float x1, y1, x2, y2, distance;

   // Prompt for coordinates of the first point
   printf("Enter x-coordinate of point 1: ");
   scanf("%f", &x1);
   printf("Enter y-coordinate of point 1: ");
   scanf("%f", &y1);

   // Prompt for coordinates of the second point
   printf("Enter x-coordinate of point 2: ");
   scanf("%f", &x2);
   printf("Enter y-coordinate of point 2: ");
   scanf("%f", &y2);

   // Calculate the distance using the distance formula
   distance = sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));

   // Print the calculated distance
   printf("The straight-line distance between the two points is: %.2f\n", distance);

   return 0;
}
```

Output:

Enter x-coordinate of point 1: 2

Enter y-coordinate of point 1: 5

Enter x-coordinate of point 2: 9

Enter y-coordinate of point 2: 3

The straight-line distance between the two points is: 7.28

Enter x-coordinate of point 1: 5

Enter y-coordinate of point 1: 6

Enter x-coordinate of point 2: 4

Enter y-coordinate of point 2: 7

The straight-line distance between the two points is: 1.41

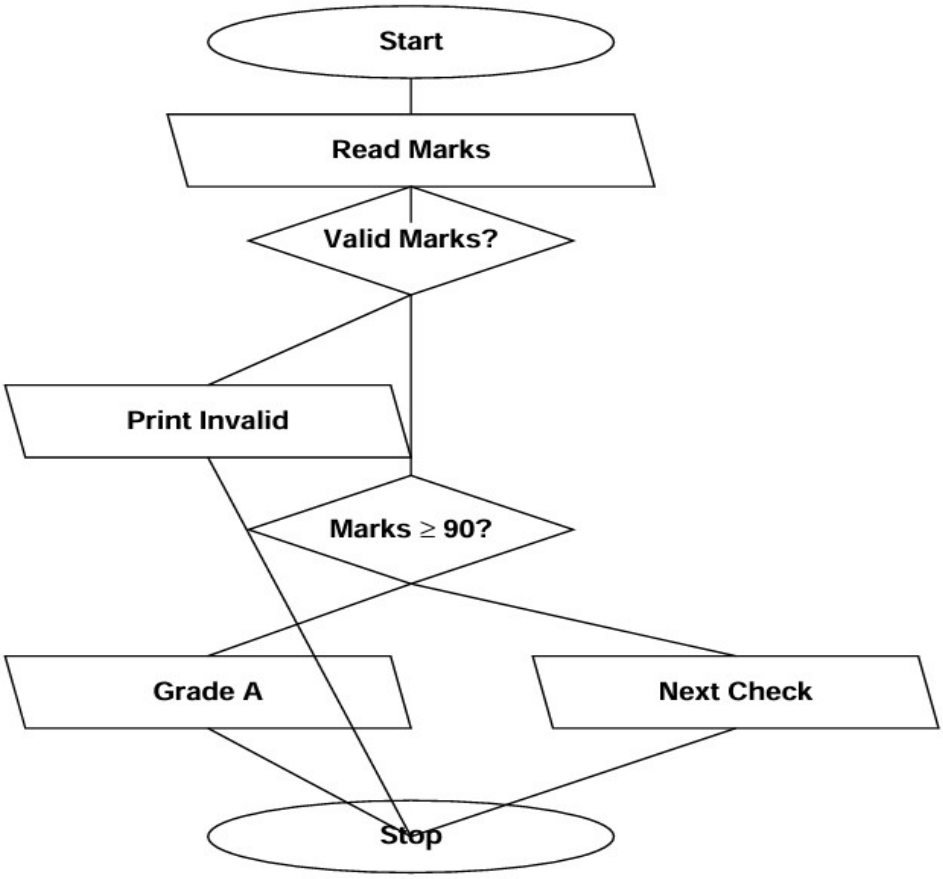
2. **Develop a C Program: that takes a student's marks as input and displays their grade based on the following criteria:**
    a. **90 and above: Grade A**
    b. **75 to 89: Grade B**
    c. **60 to74: Grade C**
    d. **50 to 59: Grade D**
    e. **Below 50: Grade F**

    Choose a suitable control structure to implement this logic efficiently.

Algorithm:

1. Start
2. Read marks
3. Validate marks
4. Decide grade using conditions
5. Display grade
6. Stop

Flowchart:



Program:

```
#include <stdio.h>
int main() {
   int marks;
   // Prompt the user to enter marks
   printf("Enter the student's marks: ");
   scanf("%d", &marks);
   // Validate marks input
   if (marks < 0 || marks > 100) {
      printf("Invalid marks entered. Marks should be between 0 and 100.\n");
   } else {
      // Determine and display the grade
      if (marks >= 90) {
         printf("Grade: A\n");
      } else if (marks >= 75) {
         printf("Grade: B\n");
      } else if (marks >= 60) {
         printf("Grade: C\n");
      } else if (marks >= 50) {
         printf("Grade: D\n");
      } else {
         printf("Grade: F\n");
      }
   }
   return 0;
}
```

Output:

Enter the student's marks: 56

Grade: D

Enter the student's marks: 89

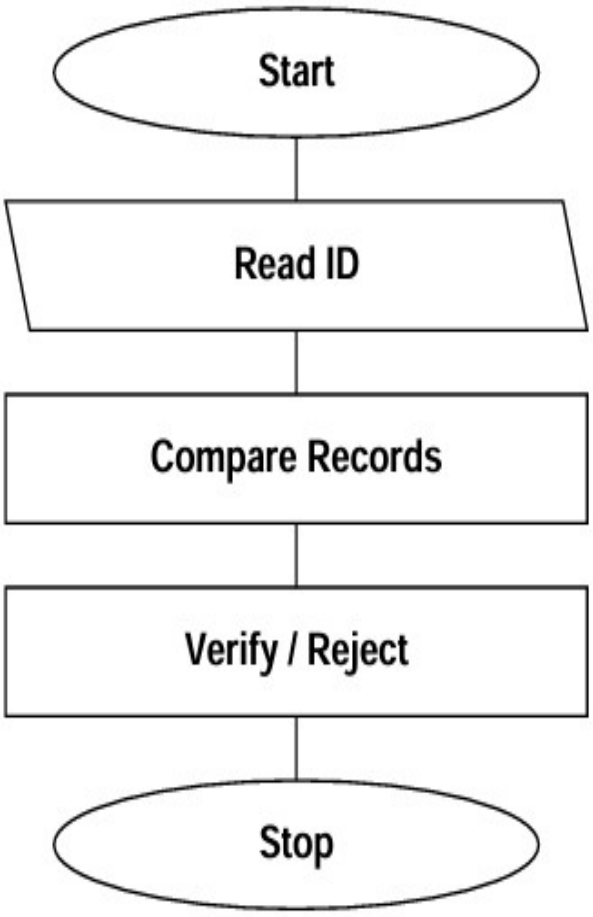
Grade: B

3. Develop a C Program: that takes a unique identification input like PAN Number, AADHAR_Number, APAAR_Id, Driving License, Passport and checks it against a set of stored KYC records. Based on the input, display whether the individual is verified or not. Use an appropriate control structure to handle multiple possible ID matches. Assume all Unique identification are of integer type.

Algorithm:

1. Start
2. Read ID
3. Compare with stored records
4. Display verification status
5. Stop

Flowchart:



Program:

```
#include <stdio.h>

int main() {
   char pan[11];
   printf("Enter PAN Number: ");
   scanf("%s", pan);
   printf("PAN Number entered: %s", pan);
   return 0;
}
```

Output:

Enter PAN Number: ABCDE1234F

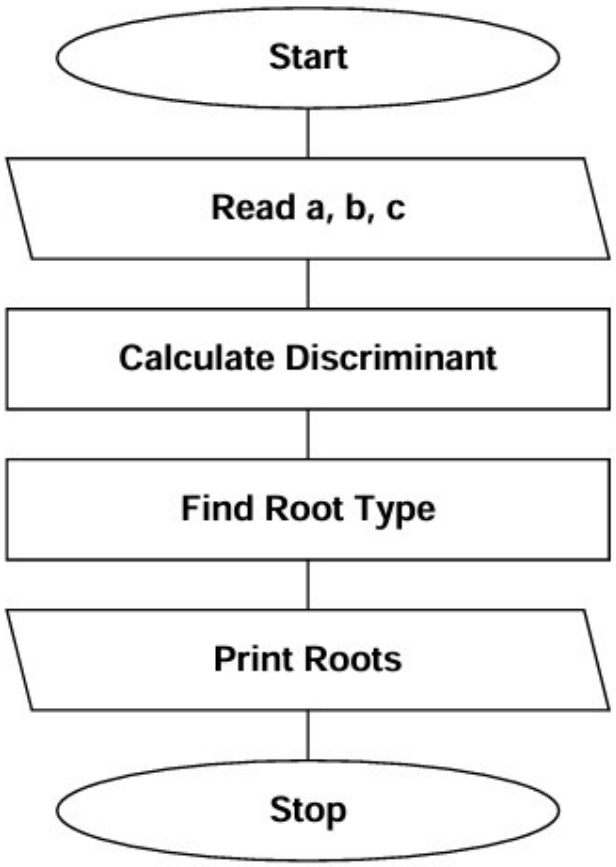
PAN Number entered: ABCDE1234F

4. **A math app needs to determine the type of roots for a quadratic equation based on user input. Develop a C Program: to calculate and display the roots based on the given coefficients.**

Algorithm:

1. Start
2. Read a, b, c
3. Calculate discriminant
4. Decide root type
5. Display roots
6. Stop

Flowchart:



Program:

```
#include <stdio.h>
#include <math.h>  // Required for sqrt() function
int main() {
    float a, b, c, discriminant, root1, root2, realPart, imaginaryPart;
    printf("Enter coefficients a, b, and c: ");
    scanf("%f %f %f", &a, &b, &c);
    discriminant = b * b - 4 * a * c;
    if (discriminant > 0) {
        root1 = (-b + sqrt(discriminant)) / (2 * a);
        root2 = (-b - sqrt(discriminant)) / (2 * a);
        printf("Roots are real and distinct: %.2f and %.2f", root1, root2);
    } else if (discriminant == 0) {
        root1 = root2 = -b / (2 * a);
        printf("Roots are real and equal: %.2f and %.2f", root1, root2);
    } else {
        realPart = -b / (2 * a);
        imaginaryPart = sqrt(-discriminant) / (2 * a);
        printf("Roots are complex: %.2f + %.2fi and %.2f - %.2fi", realPart, imaginaryPart,
realPart, imaginaryPart);
    }
    return 0;
}
```

Output:

Enter coefficients a, b, and c: 1 2 1

Roots are real and equal: -1.00 and -1.00

Enter coefficients a, b, and c: 2.3 4 5.6

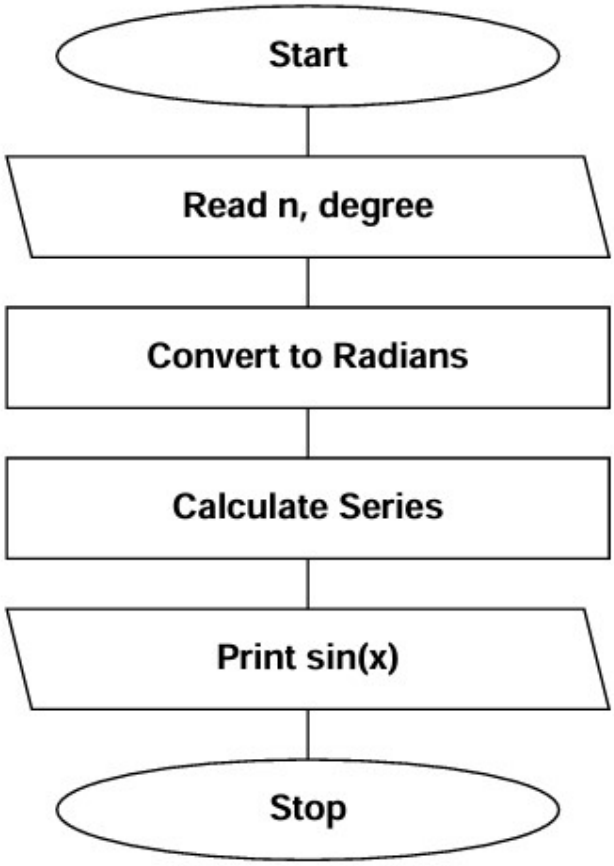
Roots are complex: -0.87 + 1.30i and -0.87 - 1.30i

5. A sensor in a robotic arm needs to calculate the angle of rotation in real-time, but the hardware doesn't support built-in trigonometric functions. Develop a C Program: to approximate the value of sin(x) using a series expansion method for improved performance.

Algorithm:

1. Start
2. Read n and degree
3. Convert to radians
4. Calculate series
5. Display result
6. Stop

Flowchart:



Program:

```
#include<stdio.h>
#include<math.h>
#define PI 3.142
int main()
{
   int n, i;
   float deg, x, sum = 0, term = 0;
   printf("Enter number of terms \n");
   scanf("%d", &n);
   printf("Enter the degree\n");
   scanf("%f", &deg);
   x = (deg * PI) / 180;
   printf("n Radians = %f \n", x);
   term = x;
   sum = term;
   for (i = 3; i <= n; i += 2)
   {
      term = (-term * x * x) / (i * (i -1));
      sum = sum + term;
   }
   printf("sin(%f)=%f\n", deg, sum);
   printf("Inbuilt function Sin(%f) = %f \n", deg, sin(x));
   printf("User function Sin(%f) = %f", deg, sum);
}
```

Output:

```
Enter number of terms
5
Enter the degree
60
n Radians = 1.047333

sin(60.000000)=0.866363

Inbuilt function Sin(60.000000) = 0.866093

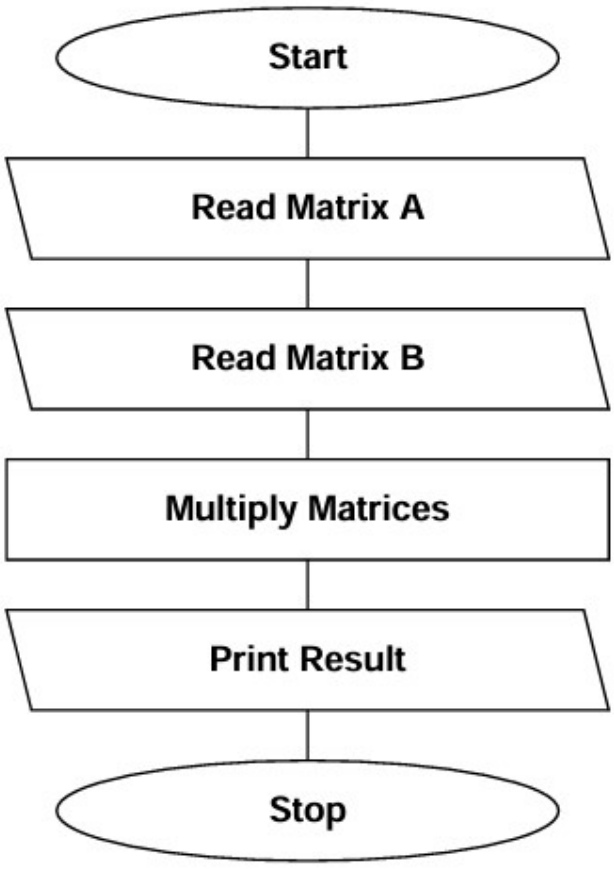
User function Sin(60.000000) = 0.866363
```

6. Implement matrix multiplication and validate the rules of multiplication.

Algorithm:

1. Start
2. Read matrices
3. Check compatibility
4. Multiply
5. Display result
6. Stop

Flowchart:



Program:

```
#include<stdio.h>
int main( )
{
 int i, j, p, q, m, n, k;
 int a[10][10], b[10][10], c[10][10];
 printf("Insert the number of rows and columns for first matrix \n ");
 scanf("%d%d", &m, &n);

 printf(" Insert your matrix elements : \n ");
 for (i = 0; i < m; i++)
  for (j = 0; j < n; j++)
   scanf("%d", &a[i][j]);

 printf("Insert the number of rows and columns for second matrix\n");
 scanf("%d%d", &p, &q);

 if (n != p)
  printf(" Given matrices cannot be multiplied with each other. \n ");
 else
 {
  printf(" Insert elements for second matrix \n ");
  for (i = 0; i < p; i++)
   for (j = 0; j < q; j++)
    scanf("%d", &b[i][j] );
  for (i = 0; i < m; i++)
   {
```

```
      for (j = 0; j < q; j++)
       {
       c[i][j]=0;
        for (k = 0; k < p; k++)
         {
          c[i][j] = c[i][j] + a[i][k] * b[k][j];
         }
       }
    }

 printf(" The result of matrix multiplication is: \n ");
    for (i = 0; i < m; i++)
     {
      for (j = 0; j < q; j++)
       {
        printf("%d\t", c[i][j] );
        }
        printf("\n ");
       }
    return 0;
}
}
```

Output:

Insert the number of rows and columns for first matrix

3 3

Insert your matrix elements
2 3 4
5 6 7
8 9 1
Insert the number of rows and columns for second matrix
3 3
Insert your matrix elements
1 2 3
3 4 6
4 5 7

The result of matrix multiplication is:

27 36 52

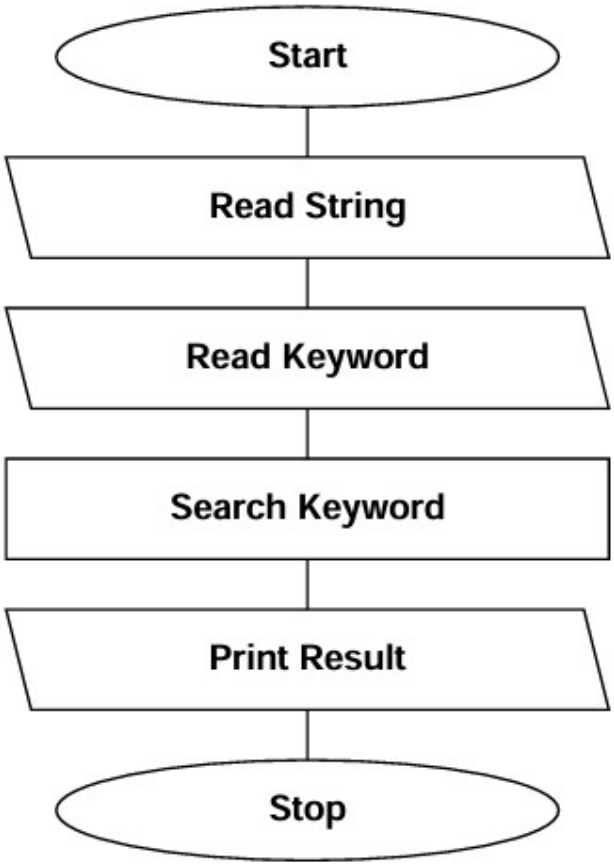
51 69 100

39 57 85

7. Develop a C Program: that accepts a course description string and a keyword from the user. Search whether the keyword exists within the course description using appropriate string functions. If found, display: "Keyword '<keyword>' found in the course description." Otherwise, display: "Keyword '<keyword>' not found in the course description."

1. Start
2. Read string
3. Read keyword
4. Search keyword
5. Display result
6. Stop

Flowchart:



Program:

```
#include <stdio.h>
#include <string.h>

int main() {
   char courseDescription[1000], keyword[100];

   // Input course description
   printf("Enter the course description: ");
   fgets(courseDescription, sizeof(courseDescription), stdin);

   // Remove newline character from the input if present
   size_t len = strlen(courseDescription);
   if (len > 0 && courseDescription[len - 1] == '\n') {
      courseDescription[len - 1] = '\0';
   }

   // Input keyword
   printf("Enter the keyword to search: ");
   fgets(keyword, sizeof(keyword), stdin);

   // Remove newline character from the keyword if present
   len = strlen(keyword);
   if (len > 0 && keyword[len - 1] == '\n') {
      keyword[len - 1] = '\0';
   }

   // Search for the keyword in the course description
   if (strstr(courseDescription, keyword) != NULL) {
      printf("Keyword '%s' found in the course description.\n", keyword);
   } else {
      printf("Keyword '%s' not found in the course description.\n", keyword);
   }

   return 0;
}
```

Output:

1. Enter the course description: C Programming in Lab
Enter the keyword to search: Lab
Keyword 'Lab' found in the course description.

2.Enter the course description: Pda College Of Engineering
Enter the keyword to search: college
Keyword 'gulbarga' not found in the course description.

Explanation:

1. **Input Handling:**
    - The fgets function is used to read the course description and keyword, ensuring spaces are handled properly.
    - Any trailing newline character from the input is removed for accurate comparison.
2. **String Search:**
    - The strstr function is used to check if the keyword exists within the courseDescription. It returns a pointer to the first occurrence of the keyword or NULL if not found.
3. **Output:**
    - If the keyword is found, a success message is displayed.
    - Otherwise, a failure message is shown.

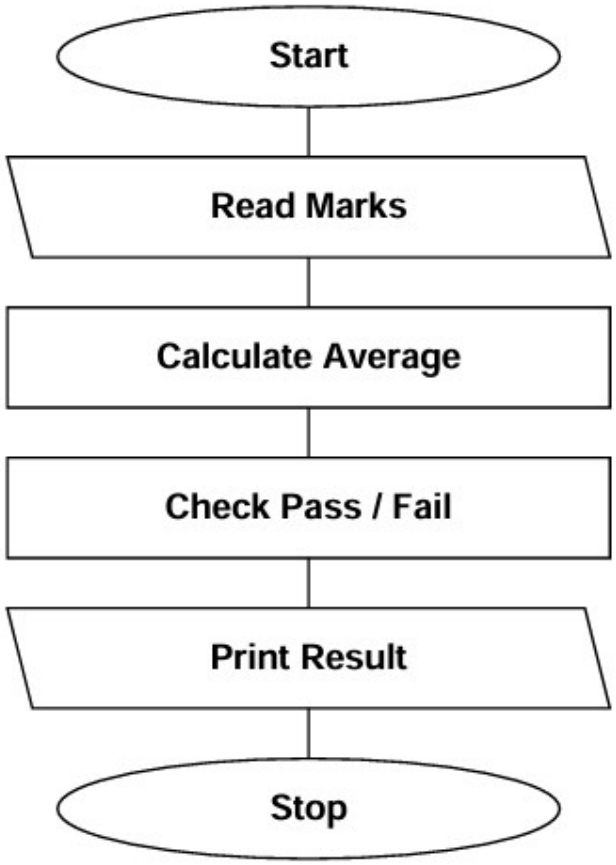
This Program: is simple, efficient, and handles user input gracefully.

8. Develop a C Program: that takes marks for three subjects as input. Use a function to check if the student has passed(minimum 40 marks in each subject). Display the average and whether the student passed or failed.

Algorithm:

1. Start
2. Read marks
3. Calculate average
4. Call function
5. Display result
6. Stop

Flowchart:



Program:

```
#include<stdio.h>

// Function to check if the student has passed
int hasPassed(int marks1, int marks2, int marks3)
 {
   return (marks1 >= 40 && marks2 >= 40 && marks3 >= 40);
}

int main() {
   int marks1, marks2, marks3;
   float average;

   // Input marks for three subjects
   printf("Enter marks for three subjects: ");
   scanf("%d %d %d", &marks1, &marks2, &marks3);

   // Calculate average
   average = (marks1 + marks2 + marks3) / 3.0;

   // Display average and pass/fail status
   printf("Average Marks: %.2f\n", average);
   if (hasPassed(marks1, marks2, marks3)) {
      printf("Status: Passed\n");
   } else {
      printf("Status: Failed\n");
   }

   return 0;
}
```

Output:

1. Enter marks for three subjects: 90
85
92
Average Marks: 89.00
Status: Passed

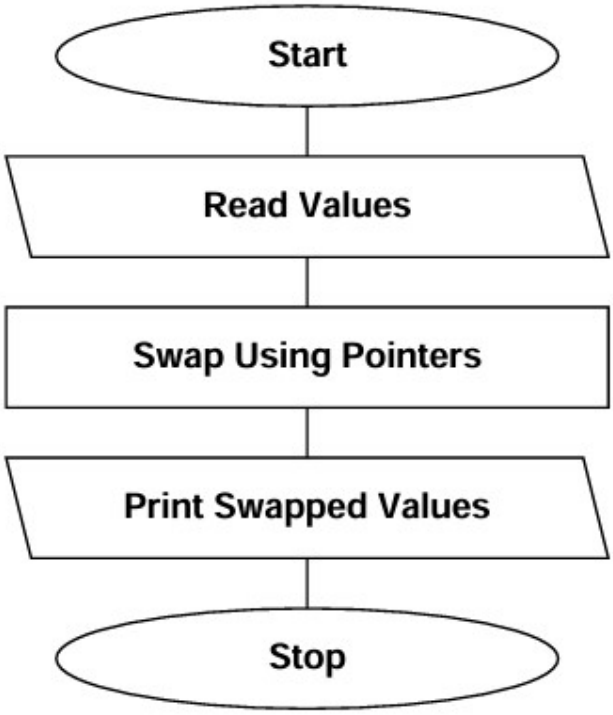
2.Enter marks for three subjects: 30
56
78
Average Marks: 54.67
Status: Failed

9. In an ATM system, two account balances need to be swapped temporarily for validation. Develop a C Program: that accepts two balances and uses a function with pointers to swap them. Display the balances before and after swapping.

Algorithm:

1. Start
2. Read balances
3. Swap using pointers
4. Display result
5. Stop

Flowchart:



Program:

```
#include <stdio.h>

// Function to swap two integer values using pointers
void swapBalances(int *balance1, int *balance2) {
   int temp; // Temporary variable to hold a value during swapping
   temp = *balance1; // Store the value pointed to by balance1 in temp
   *balance1 = *balance2; // Assign the value pointed to by balance2 to the location pointed to by balance1
   *balance2 = temp; // Assign the value in temp (original value of balance1) to the location pointed to by balance2
}

int main()
{
   int accountBalance1, accountBalance2;

   // Prompt the user to enter the two balances
   printf("Enter the first account balance: ");
   scanf("%d", &accountBalance1);

   printf("Enter the second account balance: ");
   scanf("%d", &accountBalance2);

   // Display balances before swapping
   printf("\nBalances before swapping:\n");
   printf("Account Balance 1: %d\n", accountBalance1);
   printf("Account Balance 2: %d\n", accountBalance2);

   // Call the swapBalances function, passing the addresses of the balances
   swapBalances(&accountBalance1, &accountBalance2);

   // Display balances after swapping
   printf("\nBalances after swapping:\n");
   printf("Account Balance 1: %d\n", accountBalance1);
   printf("Account Balance 2: %d\n", accountBalance2);

   return 0;
}
```

Output:

```
Enter the first account balance: 10000
Enter the second account balance: 200

Balances before swapping:
Account Balance 1: 10000
Account Balance 2: 200

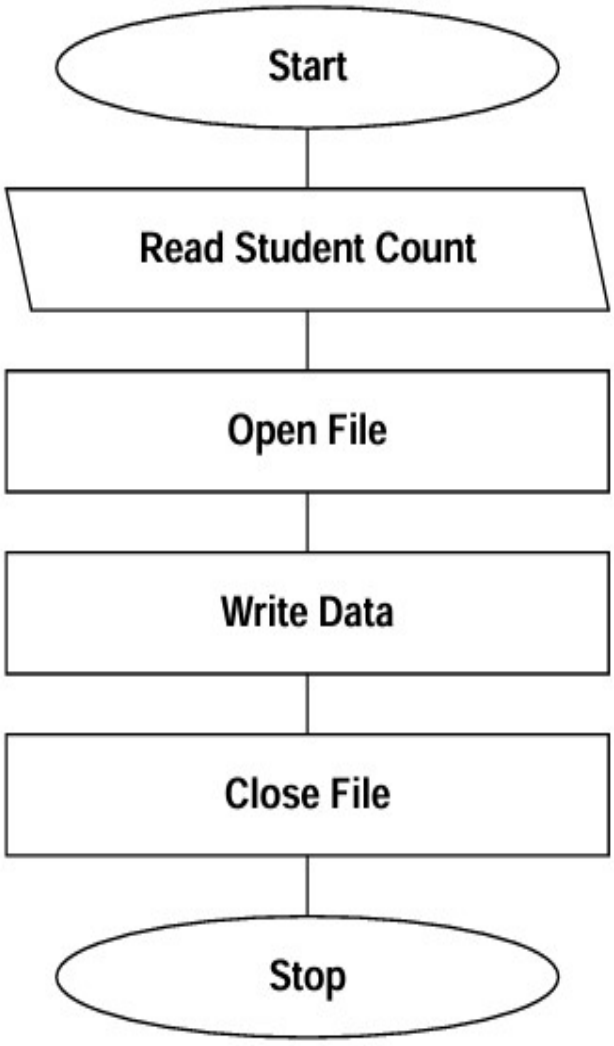
Balances after swapping:
Account Balance 1: 200
Account Balance 2: 10000
```

10. Write a C Program: to copy a text file to another, read both the input filename and target filename.

Algorithm:

1. Start
2. Read number of students
3. Open file
4. Write data
5. Close file
6. Stop

Flowchart:



Program:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
  char name[50];
  int marks, i, num;

  printf("Enter number of students: ");
  scanf("%d", &num);

  FILE *fptr;
  fptr = (fopen("student.txt", "w"));
  if(fptr == NULL)
  {
     printf("Error!");
     exit(1);
  }

  for(i = 0; i < num; i++)
  {
    printf("For student %d\n Enter name: ", i+1);
    scanf("%s", name);

    printf("Enter marks: ");
    scanf("%d", &marks);

    fprintf(fptr,"\n Name: %s \n Marks=%d \n", name, marks);
```

```
  }

  fclose(fptr);
  return 0;
}
```

Output:

```
Enter number of students 2

For student 1

Enter name: Ramesh

Enter Marks: 69

For student 2

Enter name: Aruna

Enter Marks: 98
```

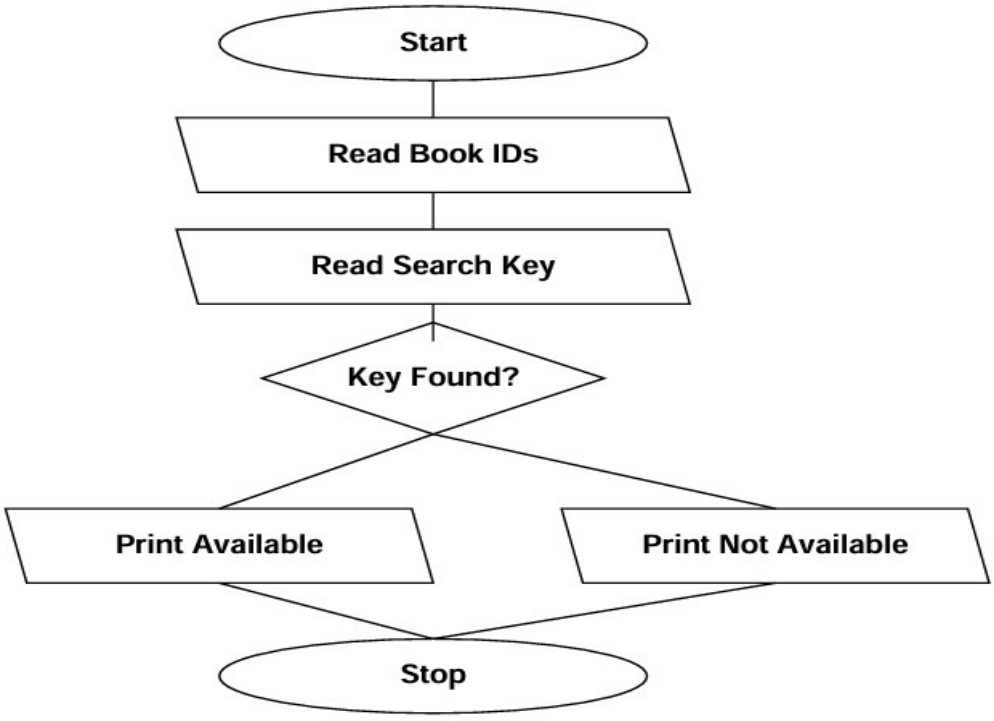
PART –B

1. A college library has a digital bookshelf system where each book is assigned a unique BookID. The bookshelf is organized in ascending order of Book IDs. Develop a C Program: to quickly find whether a book with a specific Book ID is available in the shelf.

Algorithm:

1. Start
2. Read the number of books
3. Read Book IDs into an array
4. Read the search Book ID
5. Compare the search ID with each Book ID
6. If a match is found, display **"Book Available"**
7. If no match is found after checking all IDs, display **"Book Not Available"**
8. Stop

Flowchart:



Program:

```
#include<stdio.h>
int main( )
{
int i,n, a[10], key, flag=0;
printf ("Enter no. of  Books : \n" );
scanf("%d", &n);
printf("Enter the Book ids\n");
for(i=1; i<=n; i++)
{
scanf("%d", &a[i]);
}
printf ("The given list of  Book_ids is \n");
for(i=1;i<=n;i++)
{
printf("%d ", a[i]);
}
printf("\nEnter the book id number to be searched \n");
scanf("%d", &key);
for(i=1; i<=n; i++)
{
if(key == a[i])
flag = 1;
}
if(flag==1)
printf ("The book is available \n");
else
printf("The book not available \n");
}
```

Output:

Enter no. of Books:

5

Enter the Book ids

1000

1001

1002

1003

1004

The given list of Book_ids is

1000 1001 1002 1003 1004

Enter the book id number to be searched

1003

The book is available

Enter no. of Books :

5

Enter the Book ids

1001

1002

1003

1004

1005

The given list of Book_ids is

1001 1002 1003 1004 1005

Enter the book id number to be searched

1000

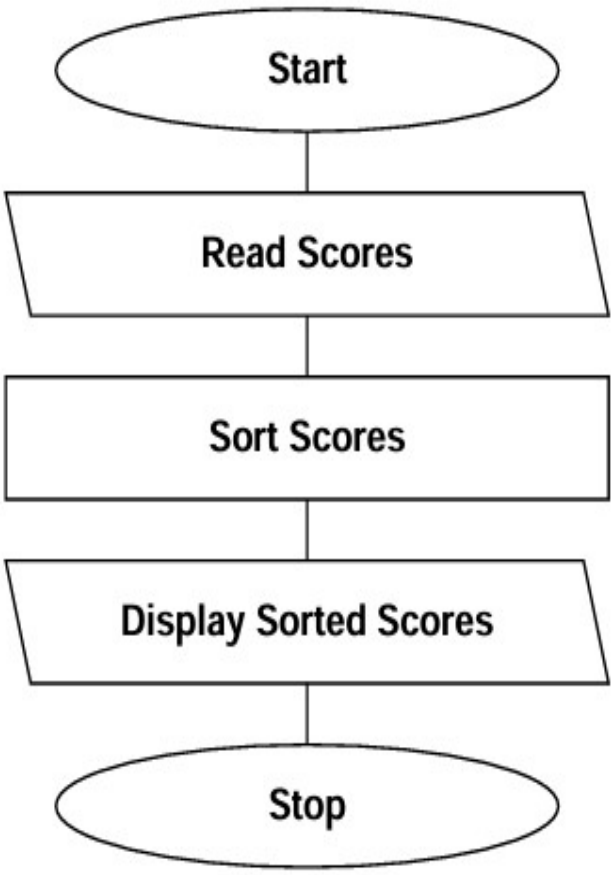
The book not available

2. A sports teacher has recorded the scores of students in a 100-meter race. To prepare the result sheet, the teacher wants the scores arranged in descending order (from highest to lowest). Develop a C Program: to sort the scores.

Algorithm:

1. Start
2. Read the number of scores
3. Read all scores into an array
4. Sort the scores in descending order using comparison and swapping
5. Display the sorted scores
6. Stop

Flowchart:



Program:

```
#include <stdio.h>
 int main() {
    int n,i,j,temp,a[100];
   printf("Enter the number of students: ");
    scanf("%d", &n
   printf("Enter the scores for the %d students:\n",n);
    for (int i = 0; i < n; i++)
{
  scanf("%d", &a[i]);
   }
 for ( i = 0; i < n - 1; i++)
     {
       for (j = 0; j < n - i - 1; j++)
        {
          if (a[j] < a[j + 1])
           {
             int temp = a[j];
             a[j] = a[j + 1];
             a[j + 1] = temp;
          }
       }
   }
   printf("\nScores in descending order (highest to lowest):\n");
   for ( i = 0; i < n; i++)
   {
      printf("%d\n", a[i]);
   }
 return 0;
}
```

Output:

Enter the number of students: 5

Enter the scores for the 5 students:

22

66

11

44

33

Scores in descending order (highest to lowest):

66

44

33

22

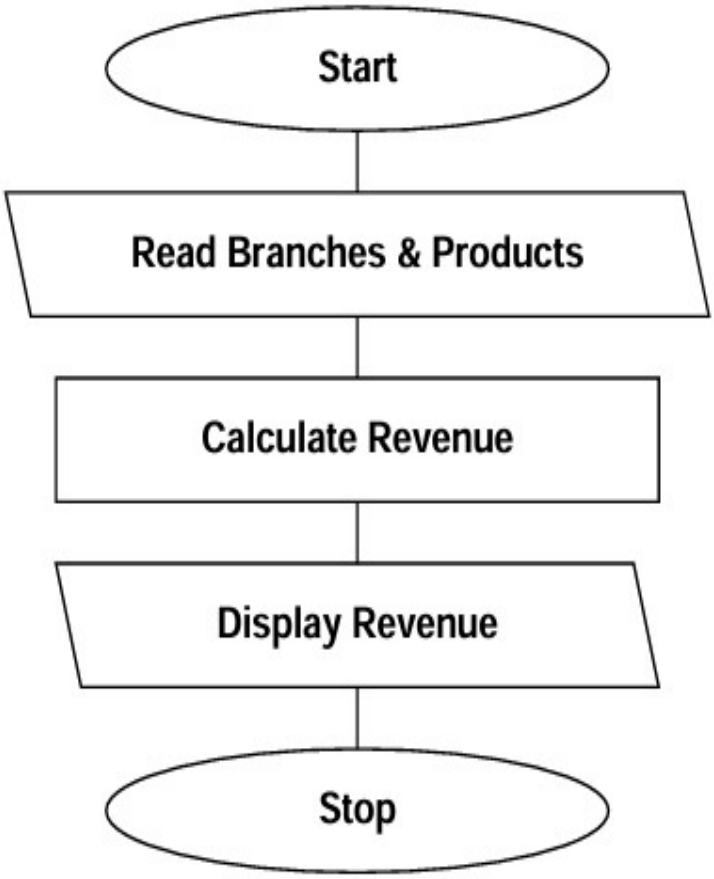
11

3. A small warehouse tracks how many units of different products are shipped from multiple branches. Another dataset shows how much revenue each product generates per unit. Develop a C Program: which combines these datasets to calculate the total revenue generated by each branch.

Algorithm:

1. Start
2. Read number of branches
3. Read number of products
4. Read sales details of each product for each branch
5. Calculate total revenue by summing sales values
6. Display the total revenue
7. Stop

Flowchart:



Program:

```
#include <stdio.h>
int main() {
    int branches, products;
    int units[10][10];
    int price[10];
    int revenue;

    printf("Enter number of branches: ");
    scanf("%d", &branches);

    printf("Enter number of products: ");
    scanf("%d", &products);

    printf("Enter price per unit of each product:\n");
    for(int i = 0; i < products; i++) {
        scanf("%d", &price[i]);
    }
    printf("Enter units shipped by each branch:\n");
    for(int i = 0; i < branches; i++) {
        for(int j = 0; j < products; j++) {
            scanf("%d", &units[i][j]);
        }
    }
    printf("\nTotal Revenue Generated by Each Branch:\n");
    for(int i = 0; i < branches; i++) {
        revenue = 0;
        for(int j = 0; j < products; j++) {
            revenue += units[i][j] * price[j];
        }
        printf("Branch %d Revenue = %d\n", i + 1, revenue);
    }

    return 0;
}
```

Output:

Enter number of branches: 2

Enter number of products: 3

Enter price per unit of each product:

50 30 20

Enter units shipped by each branch:

10 5 8

6 9 4

Total Revenue Generated by Each Branch:

Branch 1 Revenue = 810

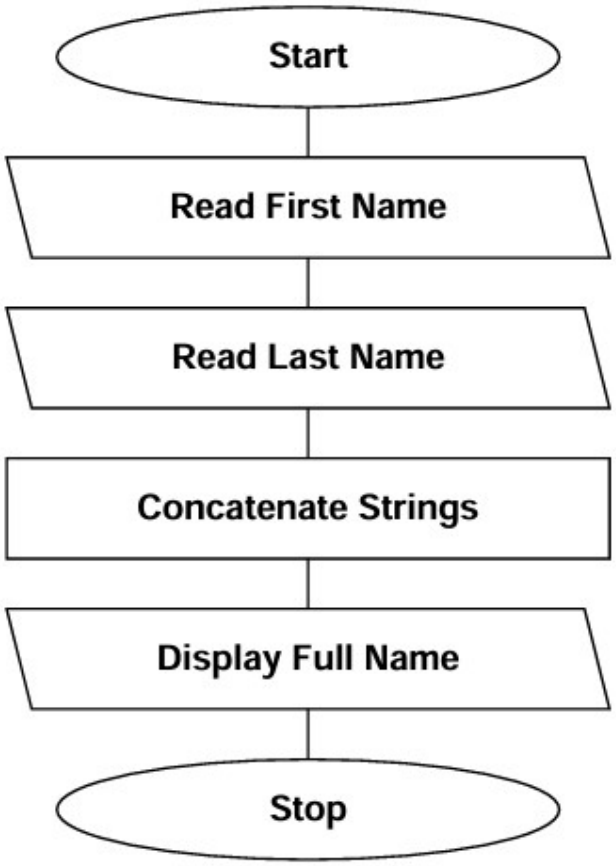
Branch 2 Revenue = 720

4. **A basic mobile contact manager stores first and last names separately. For displaying full names in the contact list, you need to join them manually. Additionally, the system must check the length of each full name to ensure it fits the screen. Perform these operations by developing a C Program: without using built- in string functions.**

Algorithm:

1. Start
2. Read the first string (first name)
3. Read the second string (last name)
4. Copy characters of the first string into a new string
5. Append characters of the second string to the new string
6. Display the concatenated string (full name)
7. Stop

Flowchart:



Program:

```
 #include <string.h>
#include<stdio.h>
 int main()
 {
        char first_name[100], last_name[100];
        int i, j;
        printf("Please Enter the First Name : ");
        scanf("%s",first_name);

        printf("\n Please Enter the Last Name:  ");
        scanf("%s”,last_name);

    // To iterate First String from Start to end
        for (i = 0; first_name[i]!='\0'; i++);

    // Concatenating Str2 into Str1
        for (j = 0; last_name[j]!='\0'; j++, i++)
        {
            first_name[i] = last_name[j];
        }
        first_name[i] = '\0';

        printf("\n After the Concatenate = %s", first_name);

        return 0;
}
```

Output:

Please Enter the First Name : xyz

Please Enter the Last Name: abc

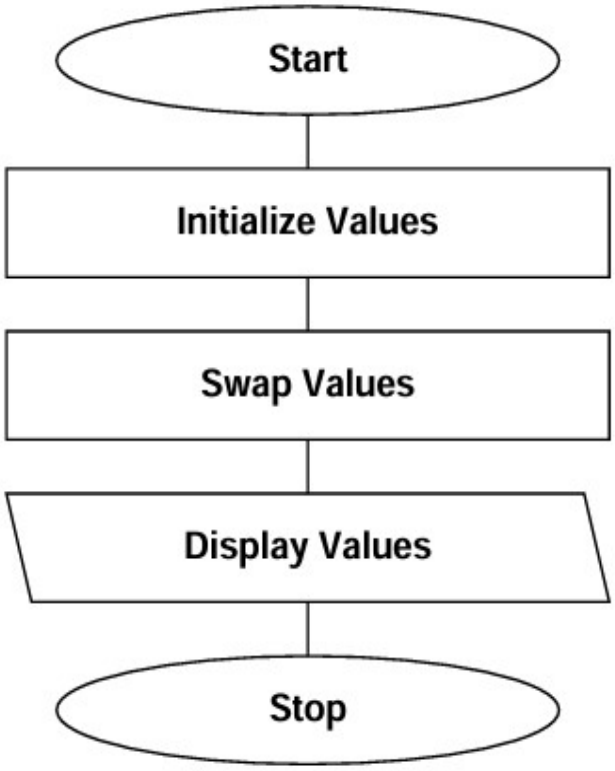
After the Concatenate = xyzabc

5. A currency exchange booth allows users to convert between two currencies. Before confirming the exchange, the system simulates a swap of the values to preview the result without actually changing the original data. In other cases, it updates the actual values. Develop a C Program: that implements both behaviors using Call by Value and Call by reference.

Algorithm:

1. Start
2. Read two integer values
3. Call the swap function by passing addresses of the variables
4. Inside the function, swap the values using a temporary variable
5. Return control to the main function
6. Display the swapped values
7. Stop

Flowchart:



Program:

```
#include <stdio.h>

/* Call by Value - Simulation only */
void simulateExchange(float usd, float eur)
{
   float temp;
   temp = usd;
   usd = eur;
   eur = temp;

   printf("\n[Simulation - Call by Value]");
   printf("\nAfter exchange:");
   printf("\nUSD = %.2f", usd);
   printf("\nEUR = %.2f\n", eur);
}

/* Call by Reference - Actual update */
void actualExchange(float *usd, float *eur)
{
   float temp;
   temp = *usd;
   *usd = *eur;
   *eur = temp;
}

int main()
{
   float usd, eur;

   printf("Enter amount in USD: ");
   scanf("%f", &usd);

   printf("Enter amount in EUR: ");
   scanf("%f", &eur);
```

```
    printf("\nOriginal Values:");
    printf("\nUSD = %.2f", usd);
    printf("\nEUR = %.2f\n", eur);

    /* Simulation using Call by Value */
    simulateExchange(usd, eur);

    printf("\nAfter Simulation (Original values unchanged):");
    printf("\nUSD = %.2f", usd);
    printf("\nEUR = %.2f\n", eur);

    /* Actual update using Call by Reference */
    actualExchange(&usd, &eur);

    printf("\n[Actual Exchange - Call by Reference]");
    printf("\nUpdated Values:");
    printf("\nUSD = %.2f", usd);
    printf("\nEUR = %.2f\n", eur);

    return 0;
}
```

Output:

Enter amount in USD: 100

Enter amount in EUR: 90

Original Values:

USD = 100.00

EUR = 90.00

[Simulation - Call by Value]

After exchange:

USD = 90.00

EUR = 100.00

After Simulation (Original values unchanged):

USD = 100.00

EUR = 90.00

[Actual Exchange - Call by Reference]

Updated Values:

USD = 90.00

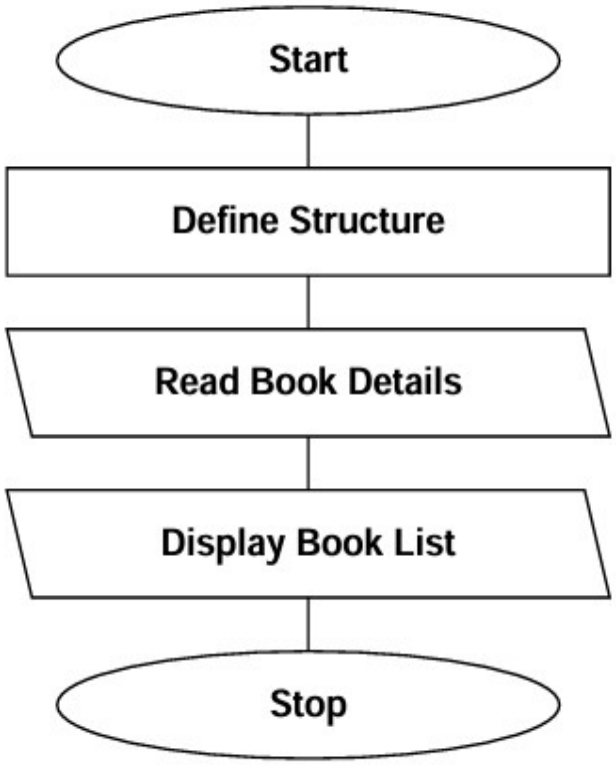
EUR = 100.00

6. A local library needs to store and display details of its books, including title, author, and year of publication. Design a structure that can hold these details and develop a C Program: to display a list of all books entered.

Algorithm:

1. Start
2. Define a structure to store book details (Book ID, Name, Author, Price)
3. Read the number of books
4. Read details of each book using the structure
5. Display all book details
6. Stop

Flowchart:



Program:

```
#include <stdio.h>
int main()
{
struct book_bank
          {
           char title[25];
           char author[20];
           int yop;
          };
struct  book_bank  book[10];
int n,i;
printf("Enter how many number of books:");
scanf("%d",&n);
printf("Enter the deatial of Books:\n");
for(i=1;i<=n;i++) {
   printf("Enter the title:\n");
   scanf("%s", book[i].title);
   printf("Enter the author: \n");
   scanf("%s", book[i].author);
   printf("Enter the year of publication:\n");
   scanf("%d", &book[i].yop);
          }
 printf("\nThe list of Book entered :\n");
  for(i=1;i<=n;i++)
   {
              printf("%s\n", book[i].title);
              printf("%s\n", book[i].author);
              printf("%d\n", book[i].yop);
          }
          printf("\n");
  }
```

Output:

Enter how many number of books:2

Enter the deatial of Books:

Enter the title:

truth

Enter the author:

a

Enter the year of publication:

2024

Enter the title:

dark

Enter the author:

b

Enter the year of publication:

2025

The list of Book entered :

truth

a

2024

dark

b

2025